03 Ashutosh

The fuzzy logic controller and methodology

Fuzzy-Logic-Based QoS Control in oneM2M (Zyane & Et-Tousy, FiCloud 2026)

Your job

You have the technical core, about **6 minutes**. You own everything mathematical, plus the "why not machine learning?" slide: membership functions, the 27 rules, inference, defuzzification, the worked example, the three design principles, and the testbed / controller configuration (moved here from John's part so his stays simple). If John or Ayush gets a "how does it compute" question in Q&A, it comes to you.

Your slides

Slides 18–30.

Opening line

"Thanks Manoj. Let's open the Plan box. Inside it is a fuzzy logic controller, and the easiest way in is a question: is 69 % CPU high?"

Hand-off line

"So that is how the controller decides. John will now show you whether it actually worked."

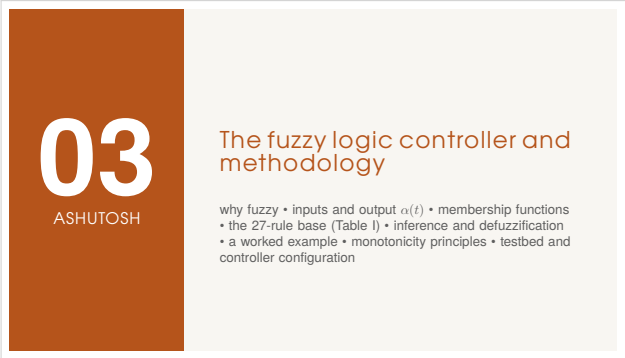
The controller in one breath

3 inputs (CPU %, RTT ms, traffic msg/s), each with 3 fuzzy sets L/M/H (trapezoids at 0/20/40/60/80/100 % of range). 27 rules cover every combination. AND = min. Rules with the same output combine with max. Output levels are spikes at VL=0, L=0.2, M=0.5, H=0.8, VH=1. Centroid = weighted average of the spikes $\Rightarrow \alpha \in [0, 1]$. $T_{\text{max}} = 10$ s, $R_{\text{max}} = 1000$ msg/s. Runs every 5 s, < 0.5 ms per run.

Worked example to know by heart

CPU 70 %, RTT 5 s, traffic 650. Memberships: CPU M 0.5 / H 0.5; RTT M 1.0; traffic M 0.75 / H 0.25. Rules 14 (M), 15 (H), 23 (VH), 24 (VH) fire at 0.5, 0.25, 0.5, 0.25. Combined: M 0.5, H 0.25, VH 0.5. $\alpha = (0.25 + 0.2 + 0.5)/1.25 =$ **0.76**.

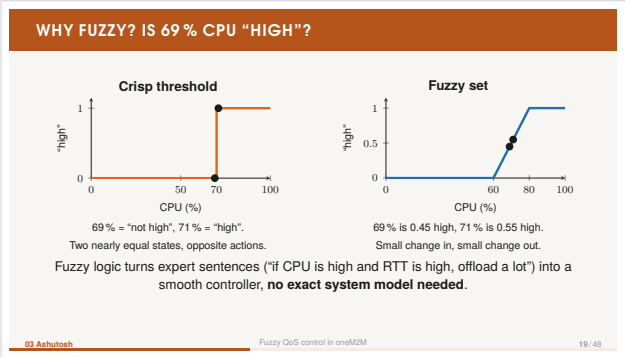
Slide 18 ~10 s



Your part opener: orange, “03 Ashutosh”.

Say: “Thanks Manoj. Let’s open the Plan box.”

Slide 19 ~40 s



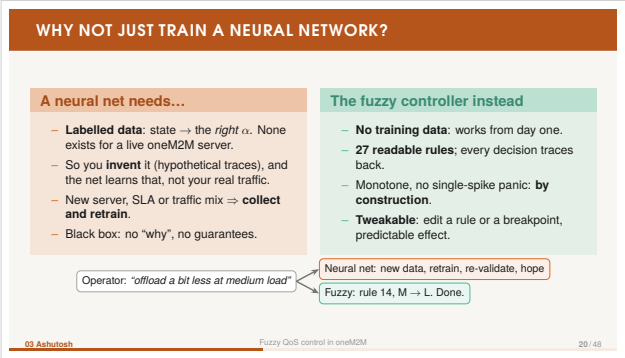
Left: a crisp step at 70 %. Right: a fuzzy ramp from 60 to 80 %. Dots at 69 and 71.

Say: “Is 69 % CPU high? With a crisp threshold at 70, 69 is ‘not high’ and 71 is ‘high’. Two almost identical states, and the system does opposite things. That is how threshold controllers end up flipping back and forth.”

Say: “A fuzzy set says 69 % is 0.45 high and 71 % is 0.55 high. A small change in the input gives a small change in the output.”

Say: “The other big advantage: you write the controller as sentences an engineer would say, like ‘if CPU is high and RTT is high, offload a lot’, and you do not need an exact mathematical model of the server.”

Slide 20 ~50 s



Why not a neural net? Left: what ML would need. Right: what fuzzy gives. Bottom: an operator’s tweak request handled both ways.

Say: “The obvious question: why not machine learning? Why not just throw a neural network at it?”

Say: “Because a neural net needs labelled data: for every combination of CPU, RTT and traffic, the *right* alpha. That dataset doesn’t exist for a live oneM2M server. So you’d have to invent one, simulated or hypothetical traces, and the network would learn that dataset, not your real traffic. And the moment you

change the server, the SLA or the traffic mix, you collect data and retrain."

Say: "Real deployments need tweakability. Say the operator says 'offload a bit less at medium load'. With a neural net: new data, retrain, re-validate, and hope. With fuzzy: you change rule 14 from medium to low, and you know exactly what changes. Properties like 'more load never means less offloading' are guaranteed by how the rules are written, not hoped for from training."

Say: "That doesn't mean learning is useless. The authors' own future work uses reinforcement learning to *tune* these rules, not to replace them. Learning adjusts the knobs, and humans can still read and override the rules."

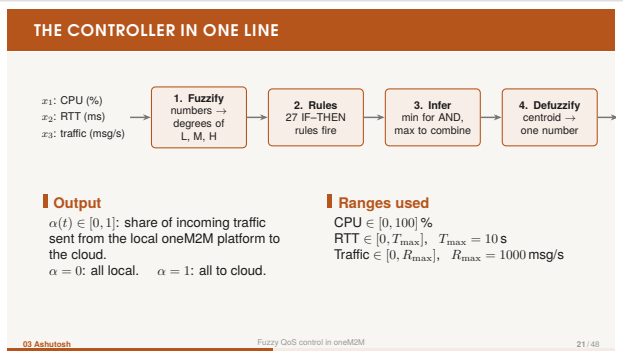
Q: *But a neural net could be more accurate?*

A: On the dataset it was trained on, maybe. Here there's no ground-truth "correct" alpha to be accurate against, the system changes over time, and an operator needs to understand and adjust the behaviour. Accuracy on a hypothetical dataset doesn't help with that.

Q: *What about reinforcement learning, which doesn't need labels?*

A: RL learns from trial and error on the live system, which means exploring bad decisions on production traffic, and it is still hard to inspect. Fuzzy Q-learning [7] is in Ayush's comparison: it works, but needs training. The paper's position is: start with readable rules, use RL later to tune them.

Slide 21 ~35 s



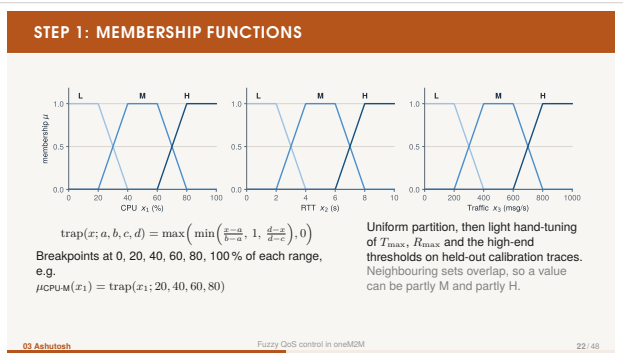
Pipeline: inputs → fuzzify → rules → infer → defuzzify → α. Output and ranges in two blocks.

Say: "The controller has three inputs: CPU usage in percent, round-trip time in milliseconds, and incoming traffic in messages per second. One output: alpha, the share of traffic sent to the cloud, between 0 and 1."

Say: "It works in four steps. Fuzzify: turn each number into degrees of low, medium, high. Rules: 27 if-then rules fire. Infer: combine them. Defuzzify: squeeze the result back into one number."

Say: "RTT is measured up to $T_{\max}$, 10 seconds, and traffic up to $R_{\max}$, 1000 messages per second, the worst cases seen in stress tests."

Slide 22 ~40 s



The nine input membership functions (our plot) and the trapezoid formula.

Say: "Each input gets three overlapping trapezoids: low, medium, high. They are placed at fixed fractions

of the range: low is fully on up to 20 % and gone by 40; medium rises from 20 to 40, stays flat to 60, and falls by 80; high starts at 60 and is full from 80.”

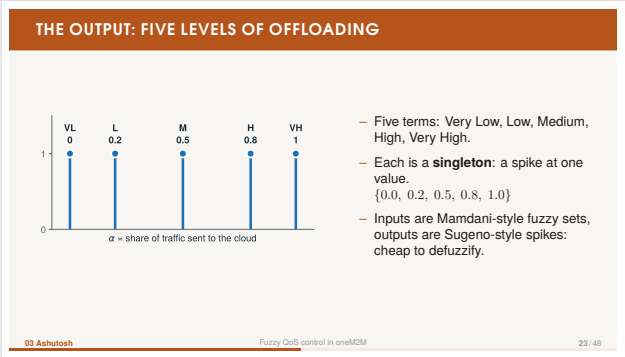
Say: “So for CPU, medium is ‘trap of 20, 40, 60, 80’. The same shape is used for RTT on 0 to 10 seconds and traffic on 0 to 1000.”

Say: “Because neighbours overlap, a value can be partly medium and partly high at the same time. That overlap is what makes the controller smooth.”

Q: *How were the breakpoints chosen?*

A: A uniform split of each range, then light manual tuning of $T_{\max}$, $R_{\max}$ and the high-end thresholds on held-out calibration traces. Automatic tuning is future work.

Slide 23 ~25 s



Five output spikes at 0, 0.2, 0.5, 0.8, 1.

Say: “The output has five levels, very low to very high. Each is a single spike, a ‘singleton’: very low means alpha 0, low 0.2, medium 0.5, high 0.8, very high 1.”

Say: “Fuzzy sets on the inputs, spikes on the output. It makes the last step a simple weighted average.”

Q: *Is it Mamdani or Sugeno?*

A: Mamdani-style rules and min–max inference, but with Sugeno-style singleton outputs. With singletons, centroid defuzzification becomes a weighted average of the five values, so it is effectively a zero-order Sugeno system.

Slide 24 ~50 s

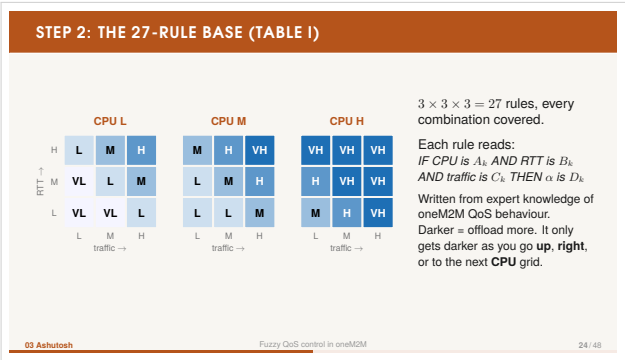


Table I as three 3×3 grids, one per CPU level; rows = RTT, columns = traffic; darker = more offloading.

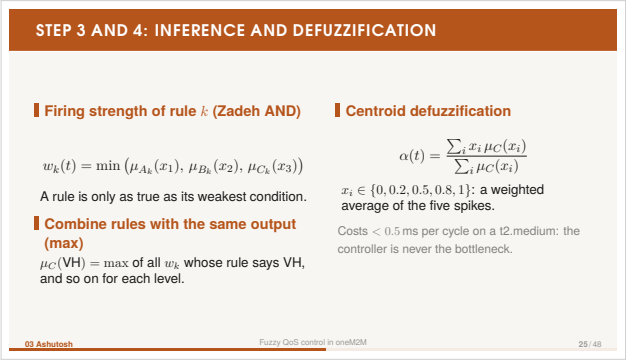
Say: “Three inputs, three levels each, gives 3 times 3 times 3, 27 rules: every combination is covered. Each rule reads: if CPU is this, and RTT is this, and traffic is this, then alpha is that.”

Say: “I’ve drawn the paper’s Table I as three grids, one for each CPU level. In each grid, RTT goes up and traffic goes right. Look at the colour: it only ever gets darker as you move up, right, or to the next grid. More stress never means less offloading.”

Say: “Bottom-left of the first grid, everything low: very low, keep everything local. Top-right of the last grid, everything high: very high, send it all to the cloud.”

Tip: Point at the two corners as you say them.

Slide 25 ~40 s



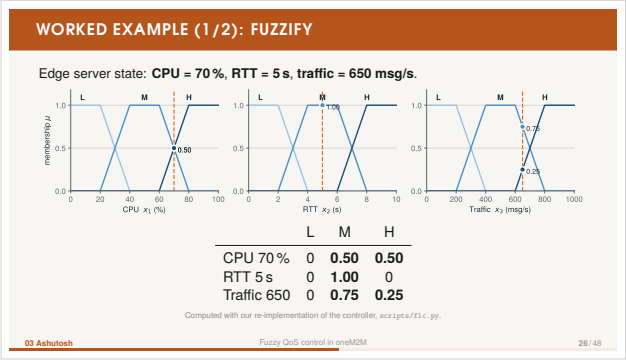
Firing strength = min; combine with max; centroid formula.

Say: "How strongly does a rule fire? The AND is a minimum: a rule is only as true as its weakest condition."

Say: "Several rules may point to the same output level; we keep the strongest, that's the max. Then the centroid: multiply each output value by its strength, add up, divide by the total strength. A weighted average."

Say: "This whole thing takes under half a millisecond per decision, so the controller is never the bottleneck."

Slide 26 ~40 s

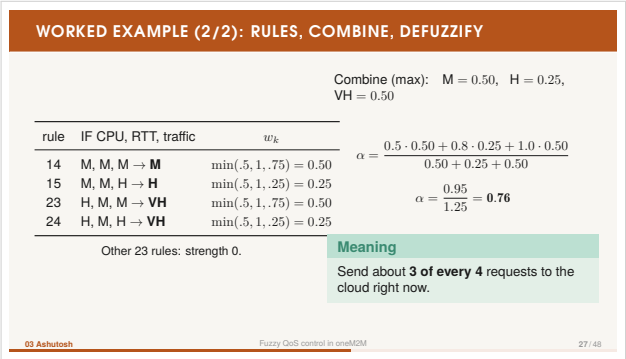


Worked example, part 1: CPU 70 %, RTT 5 s, traffic 650; orange markers on the membership plots; table of degrees.

Say: "Let's do one by hand. The edge server is at 70 % CPU, 5 seconds RTT, and 650 messages per second."

Say: "CPU 70 sits exactly between medium and high: 0.5 each. RTT 5 seconds is in the flat top of medium: fully medium, 1.0. Traffic 650 is 0.75 medium and 0.25 high."

Slide 27 ~50 s



Worked example, part 2: four fired rules, combined strengths, the division giving 0.76.

Say: "Only rules whose conditions are all non-zero fire. Here that is four rules. Rule 14, medium-medium-

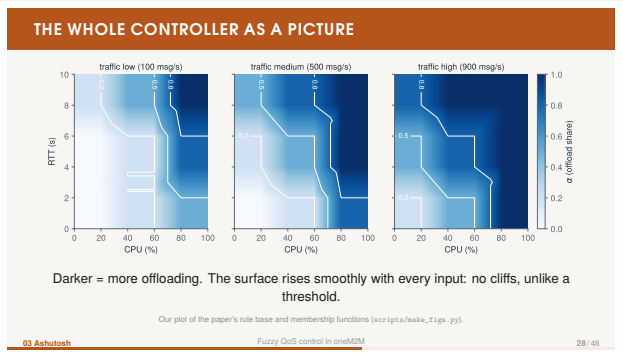
medium, says medium, with strength min of 0.5, 1, 0.75: 0.5. Rule 15 says high at 0.25. Rules 23 and 24 both say very high, at 0.5 and 0.25.”

Say: “Combine with max: medium 0.5, high 0.25, very high 0.5. Weighted average: 0.5 times 0.5, plus 0.8 times 0.25, plus 1 times 0.5, over 1.25. That is 0.95 over 1.25, **0.76**.”

Say: “So right now, send about three out of every four requests to the cloud.”

Tip: We checked this with our own Python re-implementation, `scripts/flc.py`; you can say so.

Slide 28 ~30 s

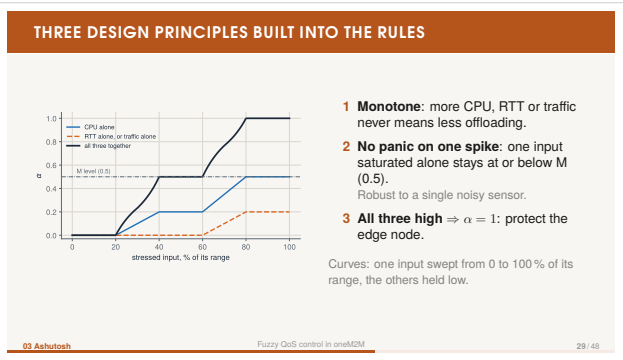


Control surface: α as a heat-map over CPU and RTT, for low, medium and high traffic.

Say: “If you run that calculation for every CPU and RTT value, you get this picture of the whole controller. Three panels for low, medium and high traffic. Darker means more offloading.”

Say: “Notice it rises smoothly in every direction. No cliffs, unlike a threshold. And as traffic increases, left to right, the whole map gets darker.”

Slide 29 ~45 s



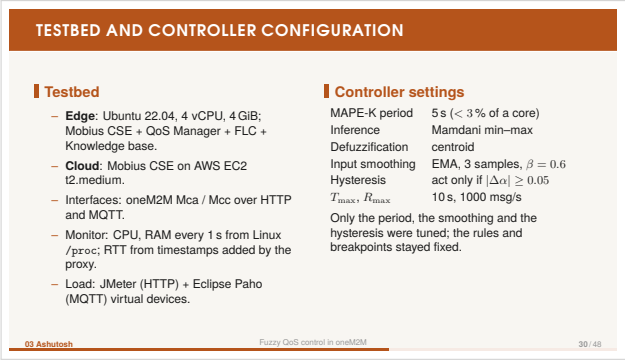
Curves of α when one input is stressed alone vs. all three together; the three principles.

Say: “The authors built three principles into the rules on purpose.”

Say: “One: monotone. More CPU, RTT or traffic never means less offloading.”

Say: “Two: no panic on one spike. Look at the coloured lines: if only CPU goes to 100 %, alpha stops at 0.5. If only RTT or only traffic saturates, it stays at 0.2. One noisy sensor cannot trigger a full offload; at least two inputs must be stressed.”

Say: “Three: the black line. When all three are high, alpha goes to 1 to protect the edge node.”



Testbed bullets on the left; controller settings table on the right.

Say: “Finally, the setup. The edge is an Ubuntu box with 4 vCPUs and 4 GB, running Mobius plus the QoS manager, the controller and the knowledge base. The cloud is Mobius on an AWS t2.medium. CPU and RAM are sampled every second from Linux’s `/proc`; RTT comes from timestamps the proxy adds. Load comes from JMeter for HTTP and Eclipse Paho for MQTT.”

Say: “The loop runs every 5 seconds, using under 3 % of a core. Inputs are smoothed with an exponential moving average, beta 0.6, so one noisy sample doesn’t jerk the controller. And there’s hysteresis: the routing only changes if alpha moves by at least 0.05.”

Say: “Only those three knobs were tuned. The rules and breakpoints stayed fixed. So that is how the controller decides. John will now show you whether it actually worked.”

Back-pocket material

Full rule base (Table I), CPU / RTT / traffic $\rightarrow \alpha$

1	L L L $\rightarrow$ VL	10	M L L $\rightarrow$ L	19	H L L $\rightarrow$ M
2	L L M $\rightarrow$ VL	11	M L M $\rightarrow$ L	20	H L M $\rightarrow$ H
3	L L H $\rightarrow$ L	12	M L H $\rightarrow$ M	21	H L H $\rightarrow$ VH
4	L M L $\rightarrow$ VL	13	M M L $\rightarrow$ L	22	H M L $\rightarrow$ H
5	L M M $\rightarrow$ L	14	M M M $\rightarrow$ M	23	H M M $\rightarrow$ VH
6	L M H $\rightarrow$ M	15	M M H $\rightarrow$ H	24	H M H $\rightarrow$ VH
7	L H L $\rightarrow$ L	16	M H L $\rightarrow$ M	25	H H L $\rightarrow$ VH
8	L H M $\rightarrow$ M	17	M H M $\rightarrow$ H	26	H H M $\rightarrow$ VH
9	L H H $\rightarrow$ H	18	M H H $\rightarrow$ VH	27	H H H $\rightarrow$ VH

Membership functions

trap$(x; a, b, c, d) =$ max(min($\frac{x-a}{b-a}, 1, \frac{d-x}{d-c}$), 0); if $a = b$ or $c = d$ that side is a vertical step.
CPU: L $= (0, 0, 20, 40)$, M $= (20, 40, 60, 80)$, H $= (60, 80, 100, 100)$.
RTT: same shape at $\times T_{\text{max}}/100$, $T_{\text{max}} = 10$ s. Traffic: same at $\times R_{\text{max}}/100$, $R_{\text{max}} = 1000$ msg/s.

Likely questions

Q: *Why min for AND, not product?*

A: Min (Zadeh) is the classic Mamdani choice and is easy to read: a rule is as true as its weakest premise. Product would give smoother but less intuitive strengths.

Q: *The paper says "single input saturation does not trigger full offloading". But rule 21 (H L H) gives VH?*

A: Right, the principle is about one input alone. Rule 21 has two inputs high (CPU and traffic), which is allowed to go above M.

Q: *Why does CPU alone reach 0.5 but RTT or traffic alone only 0.2?*

A: By design of Table I: CPU high with the others low is rule 19 $\rightarrow$ M (0.5); RTT high alone is rule 7 $\rightarrow$ L (0.2); traffic high alone is rule 3 $\rightarrow$ L (0.2). Our reading: the authors treat CPU as the most direct sign of edge overload.

Q: *Isn't there a feedback problem: offloading lowers CPU, which lowers α?*

A: Yes. The EMA and the 0.05 hysteresis damp it, but the paper does not analyse stability. Ayush shows our re-simulation of exactly this.

Q: *Why 27 rules and not fewer?*

A: Full Cartesian coverage guarantees every input state fires at least one rule. A smaller base would risk gaps where no rule fires.